NEXANOVA PROTECH

Problem Statement Document

Submitted by

Manognya Bondugula

PROTECH TRAINER MANAGEMENT SYSTEM

Project Overview

This project is a full-stack web application designed to efficiently manage trainers and subjects in an educational institute or training centre. The system supports seamless CRUD (Create, Read, Update, Delete) operations for trainers and subjects, offering a user-friendly interface for administrators.

This project includes 4 parts:

- Database designing (Tables, Formats etc.)
- Backend (Used Javascript in this project)
- API testing
- Frontend (Using HTML, CSS and others)

DATABASE DESIGNING:

The database is constructed in a table format using tools like MySQL workbench and MySQL Command Line client.

Began by setting up a MySQL database named something like protech. Within this database, we created a table called trainer with the following structure:

```
CREATE TABLE trainer (
  trainer_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100),
  phone VARCHAR(20),
  specialization VARCHAR(100),
```

```
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

This table stores each trainer's details including their name, email, phone number, and their specialization (which connects them to a subject).

This is the basic database table creation for trainer data.

BACKEND:

Here Javascript is used. The tools which are used here to construct a proper backend are:

- Node.js (Server-side JavaScript runtime)
- Express.js (Backend framework)
- Mysql2 (Node.js connector to MySQL)
- dotenv (Securely handle credentials/configs)
- cors (Allow frontend to access backend APIs)

Firstly, creating a project folder using cmd,

“mkdir protech”

“cd protech”

This creates a project folder and directs the path.

Then create a backend folder using same structure,

“mkdir backend”

“cd backend”

Next,

“npm init -y”

This sets up a package.json file for your project.

Install the required packages like,

- express

- mysql2
- cors
- dotenv

In CMD,

" npm install express mysql2 cors dotenv"

Create an .env file in the backend folder,

"type nul >.env"

Create an db.js file in the backend folder,

"type nul >db.js"

Create an server.js file in the backend folder and write:

"type nul >server.js"

Now, create a Routes folder, and create trainerRoutes.js which handles Adding a trainer, getting all trainers, by ID or subject/topic and also deleted the trainers.

"type nul >trainerRoutes.js"

Also subjectRoutes.js which handles adding subjects, handling subjects and get subject by trainer ID.

"type nul >subjectRoutes.js"

Codes are uploaded in GitHub Repository.

API TESTING:

Used Postman, a powerful API testing tool, to manually verify and test the backend functionality before connecting it to the frontend.

Here's a step-by-step description of how it worked:

1. Setting up Postman:
   Open Postman and create a new request. Set the method to POST, GET, or DELETE depending on what we were testing.
   By using the base API URL

   http://localhost:3000/api

2. Add a New Trainer (POST):
   Endpoint: POST /api/trainers
   Body(JSON):

```
{
 "name": "John Doe",
 "email": "john@example.com",
 "phone": "1234567890",
 "specialization": "AI"
}
```

   Output:

```
{
 "message": "Trainer added",
 "id": 1
}
```

Get All Trainers (GET):

Endpoint: GET /api/trainers

Expected Result:

A list of all trainer records in JSON format.

Delete a Trainer by ID (DELETE):

Endpoint: DELETE /api/trainers/1

Expected Result:

{

"message": "Trainer deleted"

}

Get Trainer by ID (GET):

Endpoint: GET /api/trainer/2

Expected Result:

Returns details of the trainer with ID 2.

Get Trainers by Subject Name (GET):

Endpoint: GET /api/trainer/AI/topic

Expected Result:

List of trainers whose specialization is "AI".

3. Tested Subject API Endpoints:

Add New Subject (POST)

Endpoint: POST /api/subject

Body (JSON):

{

"subject_name": "Machine Learning"

}

Expected Result:

{

"message": "Subject added successfully",

"id": 3

}

Get All Subjects (GET):

Endpoint: GET /api/subject

Expected Result:

List of all subjects in JSON format.

Get Subject by ID with Trainers (GET):

Endpoint: GET /api/subject/2

Expected Result:

Subject details along with trainers who teach it.

- We used Postman to test all API endpoints.
- We verified functionality like adding, listing, deleting trainers, and managing subjects
- Errors like 400 (bad request), 500 (internal server error) helped us debug.
- Once all APIs responded correctly with expected data, we moved to connect it with the frontend.

FRONTEND:

The frontend of the Problem statement was designed using HTML, CSS, and JavaScript to provide a clean, interactive interface that allows users to manage trainers and subjects effectively.

1. **HTML STRUCTURE**

   created a single-page application (index.html) divided into clearly defined feature blocks for each operation:

   - Add Trainer
   - Delete Trainer

- Get Trainers by Subject Name/ID
- Add Subject
- Display All Trainers

Each block was laid out using HTML div elements and form tags with proper input fields and buttons. Semantic tags and placeholder text were used for better UX.

2. **CSS STYLING**

Custom styles were applied through style.css to enhance the appearance:

 Responsive grid layout to organize features.
 Modern styling with **soft shadows**, **rounded corners**, and **hover effects**.
 Feedback messages (like "✅ Added!", "🗑️ Deleted!") displayed in visually distinct alert boxes styled with color codes for success, error, and info.
 Fonts and paddings adjusted for readability and clean visual hierarchy.

3. **JAVASCRIPT FUNCTIONALITY**

The logic for frontend interaction was handled using vanilla JavaScript (script.js). Event listeners were added to form submissions and buttons.

- fetch() API was used to communicate with the backend for each operation:
- POST requests for adding trainers and subjects.
- GET requests to fetch trainers by subject name or ID.
- DELETE request to remove trainers by ID.

Show success or error messages immediately under each feature. Update the trainer display section after any operation. Clear form inputs after actions.

4. **USER FEEDBACK**

We focused heavily on providing real-time feedback to users:

- Upon adding a trainer:
  Message: " Added! Name: ..., Email: ..., etc."
- Upon deleting a trainer:
  Message: " Deleted! Trainer with ID ... removed successfully."
- Upon adding a subject:
  Message: " Subject Added! Subject Name: ..."

All messages appear directly under the corresponding feature block, improving user flow and visibility.

5. TESTING AND DEBUGGING

The frontend was tested in a browser while backend APIs were tested using **Postman**. Console logs and **fetch()** error handling helped in debugging API failures or misconfigurations.